IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

Yenepoya Institute of Arts , Science , Commerce & Management

Project – High Level Design

on

NATURAL LANGUAGE INTERFACE FOR E-COMMERCE INFRASTRCUTRE

Student Name: **ARYAN VIMOD**

Industry Mentor: Mr.Sumit Kumar **Guided by: Mr.Praveen**

Table of Contents

- Introduction
  - Scope of the document.
  - Intended Audience
  - System overview.
- System Design.
  - Application Design
  - Process Flow.
  - Information Flow.
  - Components Design
  - Key Design Considerations
  - API Catalogue.
- Data Design.
  - Data Model
  - Data Access Mechanism
  - Data Retention Policies
  - Data Migration
- Interfaces
- State and Session Management
- Caching
- Non-Functional Requirements
  - Security Aspects
  - Performance Aspects
- References

INTRODUCTION

This document presents the High Level Design (HLD) of the SmartShop CLI-based E-commerce Chatbot developed using Python. The system is designed to simulate an automated customer support assistant that interacts with users through a command-line interface.

The chatbot provides responses to common E-commerce queries such as product browsing, cart management, order tracking, delivery information, and customer support. It uses a rule-based approach with keyword matching and conditional logic, making it efficient, lightweight, and suitable for basic automation scenarios.

This project serves as a foundational model for understanding chatbot architecture before moving to advanced AI-based systems.

1.1 Scope of the Document

This document covers:

- Overall architecture and system components
- Functional behavior of the chatbot
- Data handling and flow of information
- Key design considerations and constraints

This document does not include:

- Detailed code-level implementation
- Database or external system integrations

1.2 Intended Audience

- Students and beginners learning chatbot development
- Developers implementing CLI-based systems
- Project evaluators and academic reviewers

1.3 System Overview

The SmartShop chatbot is a rule-based conversational system that operates entirely within a CLI environment. It interacts with users by accepting text inputs and providing predefined responses based on keyword recognition.

Key Functionalities

- Product browsing by category (laptops, phones, shoes, accessories)
- Product recommendation (best-selling items)
- Shopping cart management (add, view, clear items)
- Order tracking simulation
- Customer support information (delivery, returns, payment, support contact)

The system does not rely on external APIs or databases and maintains all data in memory during runtime.

SYSTEM DESIGN

The SmartShop chatbot follows a simple rule-based architecture where user input is processed through a series of logical checks to generate appropriate responses. The system is designed as a standalone CLI application without any external dependencies.

The design ensures:

- Fast response time
- Minimal resource usage
- Easy scalability for future enhancements

2.1 Application Design

The application is divided into logical functional units to ensure clarity and modularity:

- Input Layer: Captures user input from CLI
- Processing Layer: Handles preprocessing and keyword matching
- Logic Layer: Implements decision-making using conditional statements
- Output Layer: Displays responses to the user

This layered approach improves maintainability and readability.

2.2 Process Flow

1. System initializes and displays welcome message
2. User enters input via CLI
3. Input is converted to lowercase
4. System checks for matching keywords
5. Corresponding response is generated
6. Output is displayed
7. Loop continues until exit command

2.3 Information Flow

User Input

↓

Input Processing

↓

Keyword Matching

↓

Decision Engine

↓

Response Selection

↓

Output Display

2.4 Components Design

1. Input Handler

- Captures user input
- Performs preprocessing (lowercase conversion)

2. Keyword Matching Engine

- Identifies relevant keywords in input
- Uses substring matching

3. Decision Engine

- Determines action using if-else logic
- Routes to appropriate module

4. Response Generator

- Produces predefined responses
- Handles all output formatting

5. Cart Manager

- Adds items to cart
- Displays cart contents
- Clears cart

6. Product Module

- Stores and displays product categories and items

2.5 Key Design Considerations

- Simplicity: Easy for beginners to understand
- Performance: Instant response without delays
- Scalability: Can be upgraded to NLP-based chatbot
- Maintainability: Modular structure for easy updates
- User Experience: Clear and interactive responses

2.6 API Catalogue

- No external APIs are used
- The system operates entirely offline
- All functionalities are internally handled

DATA DESIGN

The chatbot uses simple in-memory data structures to store and manage information.

- Products stored in dictionary format
- Cart stored as a list
- Responses stored as predefined strings

3.1 Data Model

Entities

- Products: Categories and items
- Cart: List of selected items

Example Structure :

```
products = {
  "laptops": ["Dell", "HP", "Lenovo"],
  "phones": ["iPhone", "Samsung"]
}
cart = []
```

3.2 Data Access Mechanism

- Data is accessed using direct dictionary lookup
- Keyword matching determines which data to retrieve
- No database queries involved

3.3 Data Retention Policies

- Data is stored only during runtime
- Cart data is lost after program exits
- No persistent storage or logging

3.4 Data Migration

- Not applicable in current system
- Future enhancement may include database integration for persistent storage

INTERFACES

The SmartShop chatbot interacts with users through a Command Line Interface (CLI), which serves as the primary communication layer between the user and the system.

User Interface

- Text-based interaction via terminal (VS Code or system shell)
- Accepts user input using keyboard
- Displays responses in real-time

Input Interface

- Accepts free-text commands from users
- Supports basic conversational phrases (e.g., “show products”, “add item”)
- Handles multiple keywords within a single query

Output Interface

- Displays structured responses in readable format
- Provides lists (products, cart items) in organized manner
- Uses simple formatting for clarity (bullets, spacing)

Interaction Characteristics

- Interactive and continuous session
- User-driven communication flow
- No graphical or voice interface

STATE & SESSION MANAGEMENT

The chatbot operates with a temporary session that exists only during runtime. It does not maintain persistent user sessions or authentication mechanisms.

Session Characteristics

- Session begins when the program starts
- Ends when the user exits the chatbot
- No login or user identity tracking

State Handling

- Maintains a temporary state using variables
- Cart data is stored during runtime
- Each user interaction is processed independently

Limitations

- No session recovery after exit
- No multi-user support
- No historical data tracking

CACHING

Caching is not implemented in this system due to its lightweight and real-time processing design.

Reason for No Caching

- Data is static and predefined
- Responses are generated instantly
- No repeated heavy computations

Future Scope

- Can introduce caching for:
    - Frequently searched products
    - Repeated queries
    - Faster response optimization in large-scale systems

NON-FUNCTIONAL REQUIREMENTS

1. Performance

- Response time should be less than 1 second
- Efficient execution with minimal processing overhead

2. Usability

- Easy-to-use command-line interface
- Clear instructions and help commands
- Beginner-friendly interaction

3. Reliability

- Handles unexpected or invalid inputs gracefully
- Ensures continuous operation without crashes

4. Scalability

- Can be extended with:
  - NLP integration
  - Database support
  - Web or mobile interface

5. Portability

- Runs on any operating system with Python installed
- No dependency on external tools or frameworks

6. Maintainability

- Modular code structure
- Easy to update or add new features

7.1 Security Aspects

The system ensures basic security as it operates locally without handling sensitive user data.

Security Features

- No storage of personal or financial data
- No external API calls or network communication
- Safe execution within local environment

Input Handling

- User input is processed as plain text
- No execution of dynamic or unsafe commands

Limitations

- No authentication or authorization
- No encryption (not required for this scope)

Future Enhancements

- Add login system
- Secure API integration
- Data encryption for sensitive information

7.2 Performance Aspects

The chatbot is optimized for speed and efficiency due to its simple architecture.

Performance Characteristics

- Instant response generation using keyword matching
- Minimal memory usage (in-memory storage only)
- Low CPU utilization

Efficiency Factors

- No database queries
- No network latency
- Direct condition-based execution

Scalability Consideration

- Performance may degrade if:
    - Number of conditions increases significantly
    - System scales without optimization

REFERENCES

- Google
- Wikipedia
- ChatGPT , Gemini etc…….